CN1 - Le pendule de Foucault - Corrigé

April 6, 2025

1 CN1 - LE PENDULE DE FOUCAULT

Objectif : *« À l'aide d'un langage de programmation, illustrer un effet lié au caractère non galiléen du référentiel terrestre. »*

Cette capacité numérique est associée à l'exercice du TD 2 : *Le pendule de Foucault.*

1.0.1 A) Préliminaire

L'expérience du pendule de Foucault a permis de mettre en évidence la rotation de la Terre sur elle-même. On considère un pendule simple, constitué d'un fil de longueur L au bout duquel on place une masse ponctuelle m, dont le point d'accroche en A est fixe dans le référentiel terrestre. Comme l'amplitude des oscillations et faible, on supposera pour simplifier que le pendule oscille dans le plan horizontal $(\vec{e_x}, \vec{e_y})$ où l'on a choisi $\vec{e_x}$ qui pointe vers l'est, et $\vec{e_y}$ vers le nord. L'axe $\vec{e_z}$ pointe vers la verticale ascendante. On néglige tout frottement. On lance le pendule, initialement à la verticale, avec une vitesse initiale v_0 suivant l'axe (Ox). On rappelle que la loi horaire du mouvement du pendule est donnée par

$$\begin{cases} x(t) &= +x_0 \cos(\Omega \sin\lambda\ t) \sin(\omega t), \\ y(t) &= -x_0 \sin(\Omega \sin\lambda\ t) \sin(\omega t), \end{cases}$$

où $x_0 = \frac{v_0}{\omega}$ est l'amplitude du mouvement, λ la latitude, et $\omega = \sqrt{\Omega^2 \sin^2\lambda + \omega_0^2}$ la pulsation des oscillations du pendule avec $\omega_0 = \sqrt{\frac{g}{L}}$ la pulsation propre du pendule.

1.0.2 B) Étude graphique

Question 1

Le pendule de Foucault présent au Panthéon à Paris, de latitude $\lambda = 45$ °, a pour longueur $L = 63$ m. Déterminer numériquement la pulsation propre ω_0 et la vitesse angulaire de rotation du plan d'oscillation $\Omega \sin\lambda$. Déterminer de même les périodes associées T_0 et T. Les comparer.

[5]:
```
import numpy as np

""" PARAMÈTRES PHYSIQUES """
g     = 9.81            # accélération de la pesanteur (en m.s^-2)
Tsid  = 86000           # jour sidéral (en s)
Omega = 2*np.pi/Tsid    # vitesse angulaire de la Terre (en rad.s^-1)
lat   = 45              # latitude (en degré)
```

```
L      = 63                # longueur du pendule (en m)

omega0    = np.sqrt(g/L)                        # pulsation propre du pendule
T0        = 2*np.pi/omega0                      # période propre du pendule
```

Question 2

Que devrait valoir la période du jour sidéral T_{sid} pour que le plan d'oscillation du pendule fasse un tour complet au bout de 20 oscillations ? Attribuer cette valeur numérique à la variable `Tsid` et définir les grandeurs ci-dessous. On utilisera cette valeur pour la suite.

[6]:
```
Tsid      = 200                                 # jour sidéral (en s)
Omega     = 2*np.pi/Tsid                        # vitesse angulaire de la Terre␣
↪(en rad.s^-1)
Omegaloc = Omega*np.sin(lat/180*np.pi)          # pulsation plan oscillation (en␣
↪rad.s^-1)
T         = 2*np.pi/Omegaloc                    # période plan oscillation (en s)
omega     = np.sqrt(omega0**2 + Omegaloc**2)    # pulsation du pendule (en rad.
↪s^-1)
```

Question 3

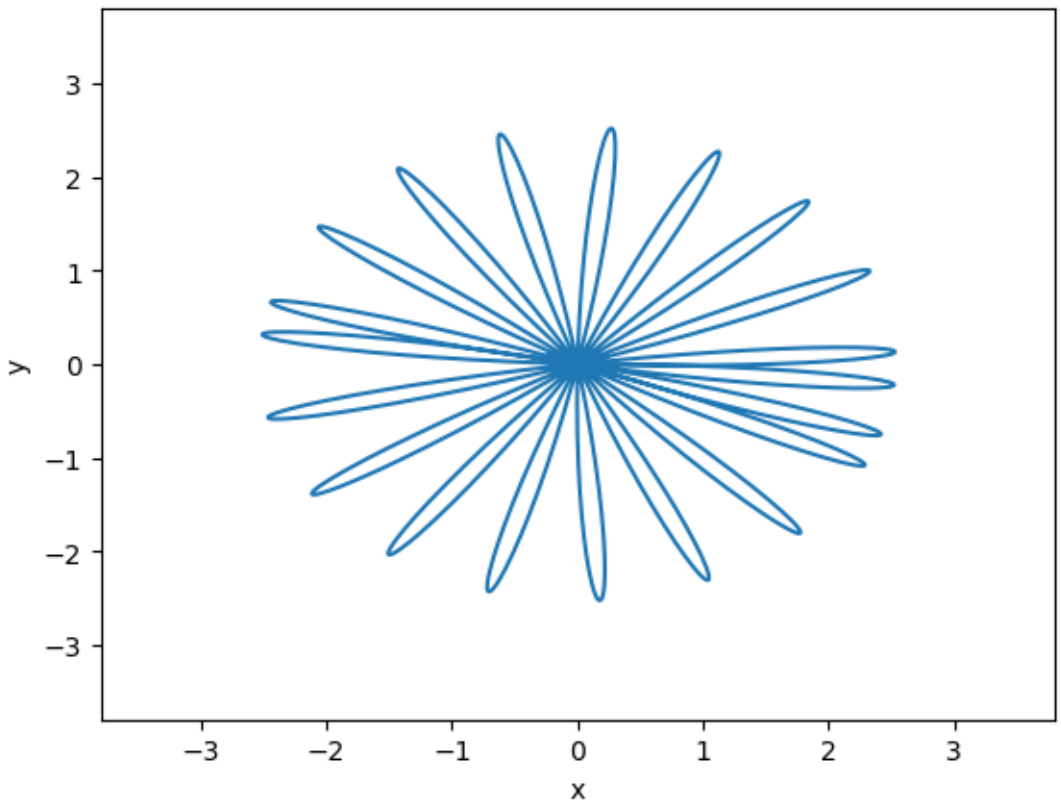
On souhaite tracer la trajectoire du pendule sur une durée équivalente à $N = 10$ périodes propres du pendule. Définir un tableau `numpy` contenant 1000 valeurs de temps échantillonnées dans cet intervalle. On utilisera la fonction `np.linspace`. Définir le tableau des positions x et y correspondant à ces instants pour une vitesse initiale $v_0 = 1\ \mathrm{m}\cdot\mathrm{s}^{-1}$.

[7]:
```
t    = np.linspace(0,10*T0,1000)              # tableau des instants de tracé
v0 = 1                                        # vitesse initiale
x0 = v0/omega                                 # amplitude du mouvement
x = +x0*np.cos(Omegaloc*t)*np.sin(omega*t)    # position x du pendule
y = -x0*np.sin(Omegaloc*t)*np.sin(omega*t)    # position y du pendule
```

Question 4

Compléter puis exécuter le code suivant de façon à produire un graphe de la trajectoire du pendule dans le plan horizontal. Observer commenter. Relancer le code en faisant varier les paramètres vitesse initiale v_0, latitude λ, etc.

[8]:
```
""" TRACÉ """
import matplotlib.pyplot as plt
plt.figure()
plt.xlim(-1.5*x0,1.5*x0)
plt.ylim(-1.5*x0,1.5*x0)
plt.xlabel('x')
plt.ylabel('y')
plt.plot(x,y)
plt.show()
```


3
2
1
0
−1
−2
−3
y
−3
−2
−1
0
1
2
3
x

[ ]:

```
```

CN2 - Corrigé

April 6, 2025

1 CN2 - RÉSOLUTION NUNÉRIQUE DE L'ÉQUATION DE DIFFUSION THERMIQUE

On s'intéresse ici à une barre métallique cylindrique de longueur $L = 1m$, dont la surface latérale est calorifugée. Les deux extrémités gauche et droite de la barre sont au contact de glace fondante de température $T_g = 0°C$, qui joue le rôle de thermostat.

L'objectif de cet exercice est de résoudre numériquement l'équation de la diffusion thermique dans cette situation simple.

1.1 A) Rappel théorique

1.1.1 Problème à résoudre

On rappelle que l'équation de diffusion thermique s'écrit

$$\boxed{\frac{\partial T}{\partial t} = \kappa \frac{\partial^2 T}{\partial x^2}}$$

où $T(t, x)$ représente la température et κ désigne la diffusivité thermique. Il s'agit d'une équation aux dérivées partielles (EDP) : * d'ordre 1 en temps t ; * d'ordre 2 en espace x.

Il donc faut adjoindre à cette équation une condition initiale $T(t = 0, x)$ et les deux conditions aux limites $T(t, x = 0) = T_g$ et $T(t, x = L) = T_g$. On considérera un matériau de coefficient de diffusivité thermique $\kappa = 10^{-4} m^2.s^{-1}$.

1.1.2 Rappel sur la méthode d'Euler

On sait résoudre analytiquement l'équation de diffusion thermique dans la plupart des situations, mais la démonstration est difficile et les fonctions utilisées font intervenir des intégrales de Gauss, qu'il est nécessaire d'évaluer numériquement dans tous les cas. Il est donc pertinent d'essayer de résoudre l'équation de la diffusion thermique de façon approchée, par une méthode **numérique**, d'autant que ces méthodes se généralisent à toute sorte d'équations différentielles. Nous utilissons ici la méthode des **différences finie**, qui est une généralisation de la méthode d'Euler vue en PCSI pour les **équations aux dérivées partielles (EDP)**.

La méthode d'Euler est une procédure numérique qui permet de résoudre de façon approximative des équations différentielles ordinaires du premier ordre avec condition initiale. Considérons l'équation

différentielle

$$\boxed{\frac{\mathrm{d}y}{\mathrm{d}t}(t) = f(y(t)).}$$

avec condition initiale $y(t=0) = y_0$ et f une fonction régulière. La résolution numérique de cette équation passe par les étapes suivantes : * discrétisation du temps t en instants t_n séparés par un pas de durée élémentaire $\mathrm{d}t$; * discrétisation de l'opérateur de dérivation $\frac{\mathrm{d}y}{\mathrm{d}t}$: on approxime la dérivée par une différence finie (taux d'accroissement)

$$\frac{\mathrm{d}y}{\mathrm{d}t} \simeq \frac{y(t_{n+1}) - y(t_n)}{\mathrm{d}t}.$$

* calcul itératif des valeurs de $y_n = y(t_n)$ d'après l'équation

$$y_{n+1} \simeq y_n + f(y_n)\mathrm{d}t.$$

La méthode d'Euler présentée ici (méthode explicite) est simple à implémenter mais présente un gros défaut de divergence de la solution numérique. En pratique, elle n'est pas utilisée pour des résolutions précises. La méthode des **différences finies** que nous allons présenter a aussi ce défaut, mais nous admettrons que la solution numérique converge bien aux temps longs avec les valeurs des paramètres que nous choisirons. Pour davantage d'information sur la **méthode d'Euler**, consulter : https://femto-physique.fr/analyse-numerique/euler.php.

1.2 B) Résolution numérique par la méthode des différences finies

ATTENTION : Nous représenterons la température par un tableau de type `numpy.array`. On aura de plus besoin du module `matplotlib.pyplot` pour tracer les graphes de la température. On importe donc ces bibliothèques python au préablable :

[1]:
```
import numpy as np
import matplotlib.pyplot as plt
```

1.2.1 Discrétisation

Soit $L = 1\,m$ la longueur de la barre et $\Delta t = 30\,min$ la durée de l'expérience. On discrétise l'espace et le temps de la façon suivante. * l'intervalle $[0, \Delta t]$ est découpé en N_t instants $t_i = i \cdot \mathrm{d}t$ séparées par un pas temporel $\mathrm{d}t = 0,4\,s$. * l'intervalle $[0, L]$ est découpé en N_x positions $x_j = j \cdot \mathrm{d}x$ séparées par un pas spatiale $\mathrm{d}x = 0,01\,m$,

Question 1

Compléter les lignes suivantes pour définir les variables `Nx` et `Nt` à partir des variables `dx`, `L`, `dt` et `Deltat`. Les durées sont exprimées en seconde et les longueurs en mètre.

ATTENTION : les variables `Nx` et `Nt` doivent être de type *int* (entier) !

[ ]:
```
dx = 1e-2
dt = 4e-1
L  = 1
Deltat = 30*60
```

```
# solution
Nx = int(L/dx) + 1
Nt = int(Deltat/dt) + 1
```

Question 2

Créer un tableau `T` de type `numpy.array` rempli de zéros, à l'aide de la fonction `np.zeros`. Chaque valeur `T[i,j]` représentera la valeur de la température $T(t,x)$ aux instants et positions $t_i = i \cdot dt$ et $x_j = j \cdot dx$. On pourra consulter le lien ci-dessous pour l'utilisation de la fonction `np.zeros` : https://numpy.org/doc/2.0/reference/generated/numpy.zeros.html.

Créer également les tableaux `t` et `x`, de type `numpy.array`, correspondant respectivement aux instants t_i et aux positions x_j pour lesquels la température est évaluée. On utilisera les variables `Nx`, `Nt`, `L` et `Deltat` définies précédemment, et on se servira de fonctions `np.linspace` : https://numpy.org/doc/2.0/reference/generated/numpy.linspace.html#numpy-linspace.

[ ]:
```
# solution
T = np.zeros((Nt,Nx))
t = np.array([i*dt for i in range(Nt)])
x = np.array([j*dx for j in range(Nx)])
```

1.2.2 Initialisation

Question 3

On suppose que le profil **initial** de température a une forme sinusoïdale, d'expression

$$\boxed{T(t=0,x) = T_g + T_0 \sin(2\pi x/L),}$$

où l'amplitude des variations de température vaut $T_0 = 20°C$ et la température de la glace fondante vaut $T_g = 0°C$.

Définir les variables `T_0` et `Tg`, puis remplir la première ligne `T[0,:]` du tableau `T` pour spécifier les conditions initiales de l'expérience. On pourra utiliser la fonction `np.sin`.

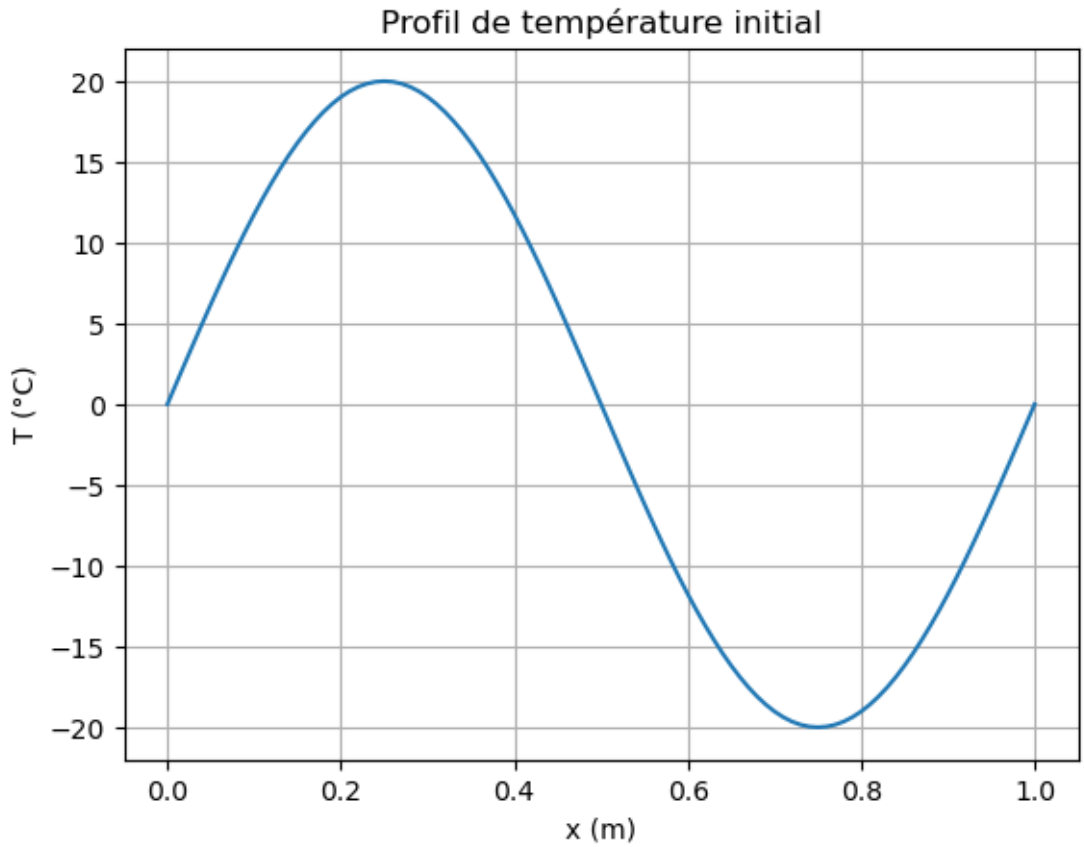
[ ]:
```
T0 = 20
Tg = 0
T[0,:] = Tg + T0*np.sin(2*np.pi*x/L) # on remplit le tableau pour i=0 (instant␣
↪initial)
```

Question 4

Compléter le code suivant pour tracer l'évolution de la température initiale au sein de la barre.

[5]:
```
plt.close()
plt.figure()
plt.grid()
```

```
plt.xlabel("x (m)")
plt.ylabel("T (°C)")
plt.title("Profil de température initial")
# solution
plt.plot(x,T[0,:])
# fin solution
plt.show()
```



1.2.3 Itérations

Question 5

Conditions aux limites. Remplir les colonnes `T[:,0]` et `T[:,-1]` tableau `T` de manière à imposer, à tout instant, les conditions aux limites aux deux extrémités de la barre, $T(x=0,t)=T_g$ et $T(x=L,t)=T_g$.

[ ]:
```
for i in range (1,Nt): # l'instant initial a déjà été traité
    T[i,0]  = Tg       # T vaut Tg=0 en x=0 à tout instant
    T[i,-1] = Tg       # T vaut Tg=0 en x=L à tout instant
```

Il s'agit donc désormais de résoudre l'équation :

$$\boxed{\frac{\partial T}{\partial t} = \kappa \frac{\partial^2 T}{\partial x^2}.}$$

Question 6

En assimilant la dérivée temporelle à un taux d'accroissement, exprimer $\frac{\partial T}{\partial t}(t = i\mathrm{d}t, x = j\mathrm{d}x)$ en fonction de `T[i,j]` et de `T[i+1,j]`.

ATTENTION : question à réaliser sur une feuille de papier !

Solution

$$\boxed{\frac{\partial T}{\partial t}(t = i\mathrm{d}t, x = j\mathrm{d}x) \simeq \frac{T[i+1,j] - T[i,j]}{dt}}$$

Question 7

Soit une fonction f. En écrivant la formule de Taylor à l'ordre 2 pour $f(x + dx)$ et $f(x - dx)$, démontrer que :

$$f''(x) \simeq \frac{f(x + dx) + f(x - dx) - 2f(x)}{dx^2}$$

ATTENTION : question à réaliser sur une feuille de papier !

Solution

On a

$$f''(x + \mathrm{d}x) = f(x) + f'(x)\mathrm{d}x + f''(x)\frac{\mathrm{d}x^2}{2} + o(\mathrm{d}x^2)$$

$$f''(x - \mathrm{d}x) = f(x) - f'(x)\mathrm{d}x + f''(x)\frac{\mathrm{d}x^2}{2} + o(\mathrm{d}x^2)$$

En sommant les deux expressions, on trouve

$$f''(x + \mathrm{d}x) + f''(x - \mathrm{d}x) \simeq 2f(x) + f''(x)\mathrm{d}x^2,$$

d'où le résultat :

$$\boxed{f''(x) \simeq \frac{f(x + dx) + f(x - dx) - 2f(x)}{dx^2}}$$

Question 8

En déduire l'approximation de $\frac{\partial^2 T}{\partial x^2}(t = i\mathrm{d}t, x = j\mathrm{d}x)$ en fonction de `T[i,j]`, de `T[i,j+1]` et de `T[i,j-1]`.

ATTENTION : question à réaliser sur une feuille de papier !

Solution

$$\boxed{\frac{\partial^2 T}{\partial x^2}(t = i\mathrm{d}t, x = j\mathrm{d}x) \simeq \frac{T[i,j+1] + T[i,j-1] - 2T[i,j]}{dx^2}}$$

Question 9

En déduire que la résolution de l'équation de diffusion thermique dans la barre se ramène au schéma numérique :

$$\boxed{T[i+1,j] = T[i,j] + \frac{\kappa \mathrm{d}t}{\mathrm{d}x^2}\left(T[i,j+1] + T[i,j-1] - 2T[i,j]\right)}$$

ATTENTION : question à réaliser sur une feuille de papier !

Solution

On remplace les dérivées dans l'équation de diffusion thermique par les différences finies calculées ci-dessus, soit

$$\frac{\partial T}{\partial t}(t = i\mathrm{d}t, x = j\mathrm{d}x) = \kappa \frac{\partial^2 T}{\partial x^2}(t = i\mathrm{d}t, x = j\mathrm{d}x) \Rightarrow \frac{T[i+1,j] - T[i,j]}{dt} \simeq \kappa \frac{T[i,j+1] + T[i,j-1] - 2T[i,j]}{dx^2},$$

d'où le résultat,

$$\boxed{T[i+1,j] = T[i,j] + \frac{\kappa \mathrm{d}t}{\mathrm{d}x^2}\left(T[i,j+1] + T[i,j-1] - 2T[i,j]\right)}$$

1.2.4 Implémentation du schéma numérique

On peut montrer qu'un tel schéma numérique converge à la condition que

$$\boxed{\frac{\kappa dt}{dx^2} < \frac{1}{2}}.$$

Avec les valeurs de dx et de dt choisies, cette condition est respectée pour la valeur choisie de la diffusivité thermique $\kappa = 10^{-4} m^2.s^{-1}$.

Question 10

Implémenter ce schéma numérique pour remplir le tableau `T` dans son intégralité. Pour cela, on calculera à chaque instant $t = i\mathrm{d}t$ et pour chaque position `j` la température `T[i+1,j]` à l'instant $t = (i+1)\mathrm{d}t = t + \mathrm{d}t$ suivant, selon l'équation trouvée en **question 9**.

[7]:
```
kappa = 1e-4

for i in range(0,Nt-1): # itération sur les instants i
    for j in range(1,Nx-1): # itération sur les positions j
        # solution
        T[i+1,j]=T[i,j]+kappa*dt/(dx**2)*(T[i,j+1]+T[i,j-1]-2*T[i,j])
        # fin solution
```

Question 11

On souhaite tracer le profil de température aux différents instants définis dans la liste `instants` ci-dessous. Compléter la boucle du suivant code de sorte à tracer sur un même

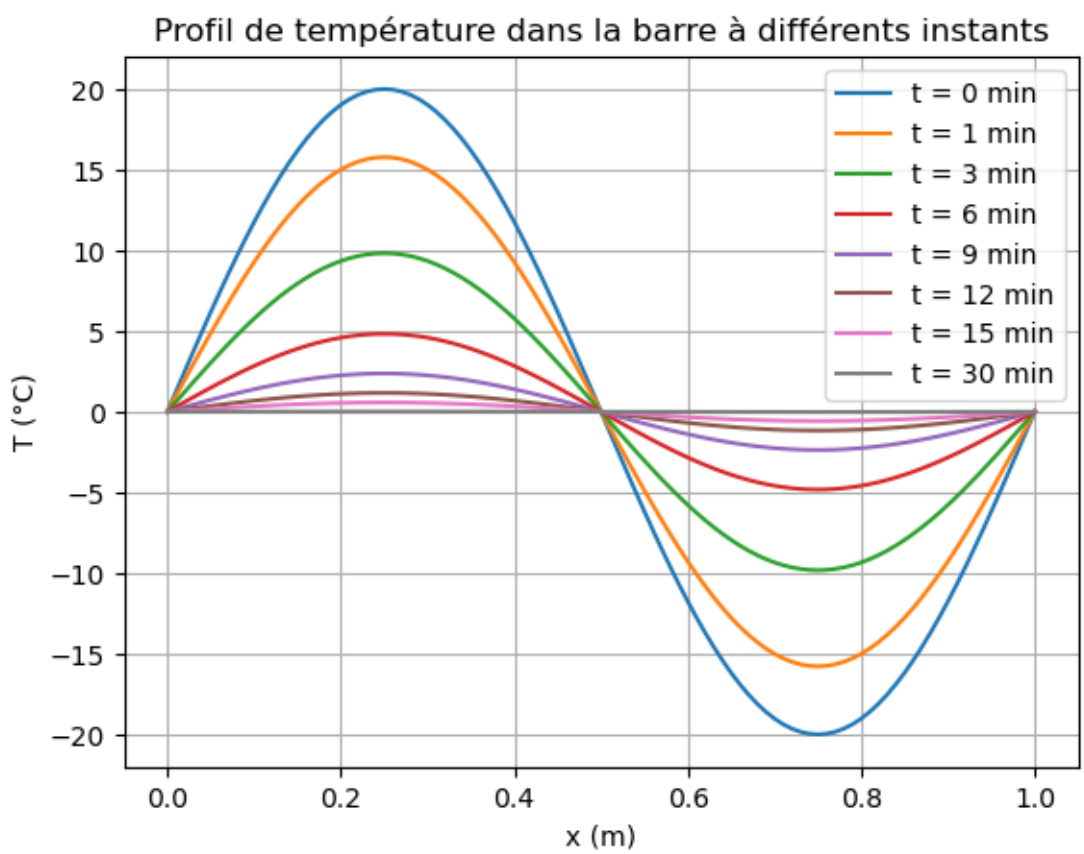
graphique l'évolution du profil de température dans le temps. Comparer la solution analytique (admise)

$$\boxed{T(t,x) = T_g + T_0 e^{-t/\tau} sin(2\pi x/L),}$$

avec $\tau = L^2/(4\pi^2\kappa)$.

[8]:
```
plt.close()
plt.figure()
plt.xlabel("x (m)")
plt.ylabel("T (°C)")
plt.title("Profil de température dans la barre à différents instants")

instants = [0,1,3,6,9,12,15,30] # instants choisis en minutes
for t in instants: # tracé du profil du température aux instants choisis
    i = int(t*60/dt)
    # solution
    plt.plot(x,T[i,:],label="t = "+str(t)+" min") # on étiquette chaque courbe␣
 ↪avec la valeur de l'instant t
    # fin solution
plt.legend()
plt.grid()
plt.show()
```



Solution

Les graphes semblent cohérents avec une décroissance exponentielle de la température dans le temps,

$$T(t,x) = T_g + T_0 e^{-t/\tau} sin(2\pi x/L).$$

Le temps typique de décroissance vaut ici $\tau = L^2/(4\pi^2\kappa) \simeq 4$ min 13 s. Cela signifie que l'amplitude de la variation spatiale de température doit diminuer d'un facteur $e \simeq 2{,}7$ au bout de $\simeq 4$ min, ce qui est cohérent avec la figure.

[ ]:

```
```

CN3 - Marche aléatoire - Corrigé

April 6, 2025

1 CN3 - MODÈLE PROBABILISTE DE LA MARCHE ALÉATOIRE

1.1 A) Rappel théorique

Un modèle de marche aléatoire permet de simuler le déplacement erratique des particules à l'échelle microscopique engendré par l'agitation thermique par exemple. On étudie une particule au milieu de particules voisines, qui se déplace au gré des chocs. On suppose pous simplifier que la particule effectue un saut de longueur ℓ à chaque pas de temps τ. Dans un modèle $1D$ (une dimension), la particule se déplace sur un axe et peut donc sauter, soit à droite, soit à gauche.

Nous avons montré en cours que la probabilité $p(x,t)$ que la particule soit à la position x à l'instant t vérifie l'équation de diffusion

$$\boxed{\frac{\partial p}{\partial t} = D\frac{\partial^2 p}{\partial x^2}},$$

où $D = \ell^2/2\tau$ est le coefficient de diffusion.

1.2 B) Marche alétoire 1D

Question 1

Nous commençons par définir les grandeurs ℓ, τ et D, puis nous simulons `Nstep` sauts de la particule. La fonction `marche` renvoie la distance au carré parcourue après `Nstep` sauts. Compléter la ligne pointillée de façon à actualiser la nouvelle position x de la particule après son saut aléatoire.

[1]:
```
""" BIBLIOTHEQUES """
import numpy as np
import random as r

""" PARAMETRES """
l   = 1             # distance des sauts
tau = 1             # durée entre sauts
D   = l**2/2/tau    # coefficient de diffusion

""" SIMULATION """
def marche(Nstep):
    x = 0                           # position initale
    for i in range(Nstep):          # boucle sur les sauts
```

```
        x = x + l*r.randint(-1,1)  # saut équiprobable à droite ou à gauche
    return x**2                    # position finale au carré
```

Question 2

On associe la distance typique de diffusion δ à la distance quadratique moyenne $\sqrt{< x^2 >} = \delta$. On souhaite calculer δ en moyennant la valeur de x^2 sur un grand nombre de simulations de marche aléatoire. Compléter le code suivant pour définir une fonction `dquad` qui prend en entrée un nombre `Nstep` de sauts à chaque simulation et renvoie la distance quadratique moyenne sur 1000 simulations.

[2]:
```
""" DISTANCE MOYENNE QUADRATIQUE """
def dquad(Nstep):
    xcarre = []                          # initialisation tableau des x^2
    for i in range(1000):                # 1000 simulations
        xcarre.append(marche(Nstep))     # simulation d'une marche
    return np.sqrt(np.mean(xcarre))      # distance moyenne quadratique
```

Question 3

On rappelle que les phénomènes diffusifs vérifient la loi

$$\boxed{\delta(t) \propto \sqrt{Dt}.}$$

En particulier, à une dimension, on peut démontrer la loi exacte

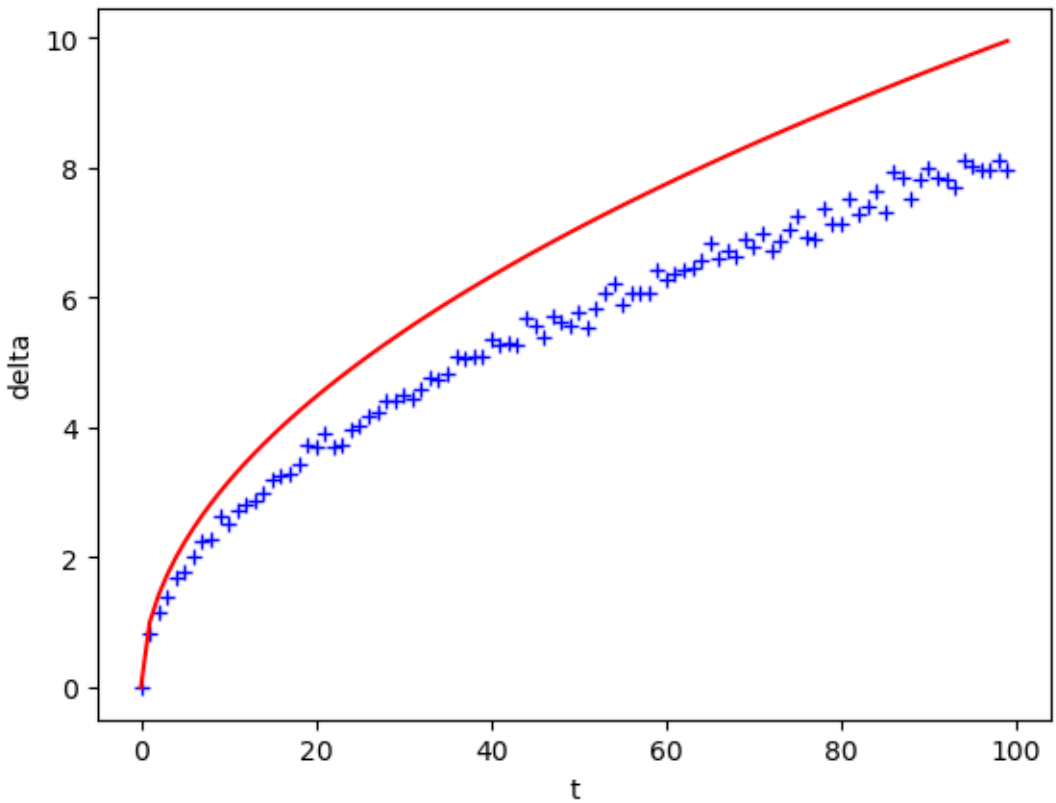
$$\delta(t) = \sqrt{2Dt}.$$

Compléter le code suivant pour représenter graphiquement les valeurs de δ pour différentes durées t de simulation. Conclure sur la validité de la loi de diffusion précédemment donnée.

[3]:
```
""" GRAPHIQUE """
import matplotlib.pyplot as plt

Nstep = np.arange(100)                          # tableau des nombres de sauts
temps = Nstep*tau                               # tableau des durées de␣
 ↪simulation
delta_diff = [np.sqrt(2*D*t) for t in temps]    # delta attendu
delta_simu = [dquad(N) for N in Nstep]          # delta simulé

plt.close()
plt.xlabel('t')                     # temps
plt.ylabel('delta')                 # delta
plt.plot(temps,delta_simu,'b+')     # delta issu de la simulation
plt.plot(temps,delta_diff,'r-')     # delta issu de la loi sqrt(2*D*t)
plt.show()                          # affichage
```



1.3 C) Marche aléatoire 2D (bonus)

Question 4 (bonus)

Simuler une marche aléatoire à deux dimensions et afficher le parcours de la particule diffusante dans le plan.

[4]:
```
# Python code for 2D random walk.
import numpy
import pylab
import random

# defining the number of steps
n = 2000

#creating two array for containing x and y coordinate
#of size equals to the number of size and filled up with 0's
x = numpy.zeros(n)
y = numpy.zeros(n)
t = 0
dt = 0.01
line, = pylab.plot(x, y)
```

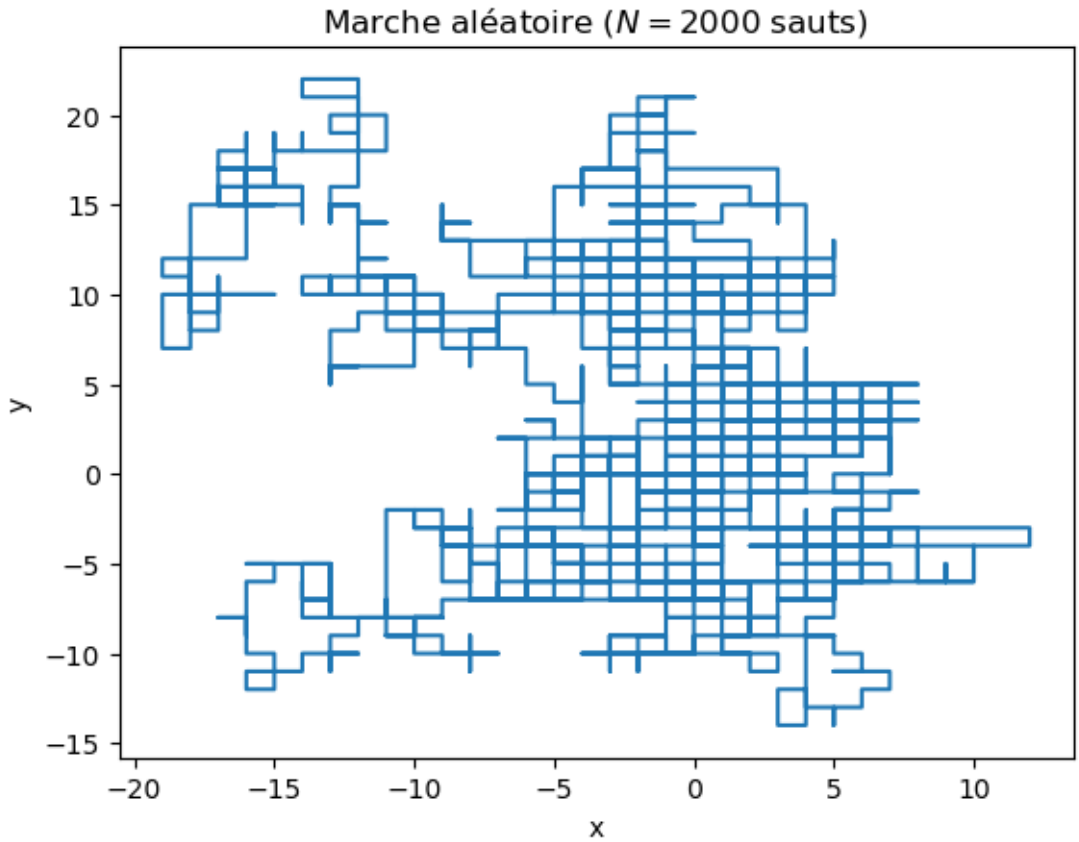
```
lim = 1.4*numpy.sqrt(n)
pylab.xlim(-lim,lim)
pylab.ylim(-lim,lim)

plt.close()
pylab.ion() # mode interaction on

# filling the coordinates with random variables
for i in range(1, n):
        t = t + dt
        val = random.randint(1, 4)
        if val == 1:
                x[i] = x[i - 1] + 1
                y[i] = y[i - 1]
        elif val == 2:
                x[i] = x[i - 1] - 1
                y[i] = y[i - 1]
        elif val == 3:
                x[i] = x[i - 1]
                y[i] = y[i - 1] + 1
        else:
                x[i] = x[i - 1]
                y[i] = y[i - 1] - 1
        line.set_ydata(y)
        line.set_xdata(x)
        #pylab.draw()
        #if i%int(numpy.log10(n)*2) == 0:
                #pylab.show()
                #pylab.pause(1/n)
xf,yf = x[-2],y[-1]

pylab.ioff()
# plotting stuff:

pylab.xlabel('x')
pylab.ylabel('y')
pylab.title("Marche aléatoire ($N = " + str(n) + "$ sauts)")
pylab.plot(x, y)
pylab.plot(xf, yf, 'r')
pylab.show()
```


Marche aléatoire ($N = 2000$ sauts)
20
15
10
5
0
−5
−10
−15
y
−20
−15
−10
−5
0
5
10
x

```
[ ]:
```

CN4 - Corrigé

April 6, 2025

1 CN4 - SIMULATION MONTE-CARLO

1.1 A) Introduction

En montage d'optique, on utilise souvent des lentilles comme système de projection. Pour extraire des données quantitatives de l'expérience, il faut donc connaître précisément la distance focale f' de la lentille. On peut pour cela mesurer les positions d'un objet A sur l'axe optique et de son image A' puis appliquer la formule de Descartes

$$f' = \left(\frac{1}{Y} - \frac{1}{X}\right)^{-1}$$

où l'on a noté $X = \overline{OA}$ la distance lentille-objet et $Y = \overline{OA'}$ la distance lentille-image .

Connaissant les incertitudes sur X et Y, peut-on déterminer l'incertitude sur f' ? Dans le TP0 sur les incertitudes, nous avons vus différentes formules de composition des incertitudes.

- Pour une somme ou une différence $Z = X \pm Y$, on a $\Delta Z = \sqrt{\Delta X^2 + \Delta Y^2}$.
- Pour un produit ou une division $Z = XY$ ou X/Y, on a $\frac{\Delta Z}{Z} = \sqrt{(\frac{\Delta X}{X})^2 + (\frac{\Delta Y}{Y})^2}$.

Mais pour la fonction de X et Y ci-dessus, vous ne connaissez pas de formule qui donne l'incertitude sur f' ! On recourt alors à une simulation de type Monte-Carlo pour étudier la variabilité de la grandeur f' sans avoir à composer d'incertitudes.

1.2 B) Préliminaire

Question 1

Définir en python une fonction `focale` qui retourne f' connaissant X et Y.

[1]:
```
def focale(X,Y):
    """
    Entrées : distance objet X, distance image Y (float)
    Sortie : distance focale f' (float)
    """
    return 1/(1/Y - 1/X)
```

Un expérimentateur a mesuré les positions d'un objet et d'une image et à trouvé $x = -15$ cm et $y = 30$ cm respectivement. En estimant sur quelle plage de positions il voit l'image nette, il a estimé que les incertitudes sur X et Y à $\Delta X = \Delta Y = \Delta = 1$ cm. Pour une simulation Monte-Carlo, nous

devons maintenant proposer une distribution de probabilités pour les variables aléatoires X et Y. On suppose cette distribution uniforme, centrée sur $x = -15$ cm et $y = 30$ cm respectivement, et de demi-largeur égale $\Delta = 1$ cm, selon le code suivant.

[2]:
```
""" Initialisations """
x     = -15   # cm, position de l'objet
y     = 30    # cm, position de l'image
Delta = 1     # cm, incertitude-type sur les positions
```

Question 2

Compléter le code ci-dessus pour réaliser $N = 100$ tirages aléatoires des grandeurs X et Y, et de stocker les valeurs de f' calculées à partir de X et Y sous forme de liste. On pourra pour cela utiliser la fonction `np.random.uniform`, dont on donne la documentation ci-dessous.

1.3 C) Simulation

La fonction `np.random.uniform` permet de générer un nombre flottant aléatoire tiré d'une distribution uniforme. Les échantillons sont répartis uniformément sur l'intervalle [low, high]. En d'autres termes, toute valeur dans l'intervalle donné a la même probabilité d'être tirée selon la distribution uniforme.

`np.random.uniform(low,high)`

Paramètres :

- `low` : float ou tableau de flottants, optionnel Limite inférieure de l'intervalle de sortie. Toutes les valeurs générées seront supérieures ou égales à low. La valeur par défaut est 0.
- `high` : float ou tableau de flottants Limite supérieure de l'intervalle de sortie. Toutes les valeurs générées seront inférieures ou égales à high. La limite supérieure peut être incluse dans le tableau de flottants retourné en raison de l'arrondi des nombres à virgule flottante dans l'équation low + (high-low) * random_sample(). La valeur par défaut est 1.0.

[3]:
```
""" Importation des bibliothèques """
import numpy as np

""" Initialisations """
N = 100  # nombre de tirages
f = []   # liste de valeurs de f'

""" Tirage de N valeurs de f' """
for i in range(N):
    X = np.random.uniform(x-Delta,x+Delta)  # tirage de X
    Y = np.random.uniform(y-Delta,y+Delta)  # tirage de Y
    f.append(focale(X,Y))                   # calcul de f'
```

Question 3

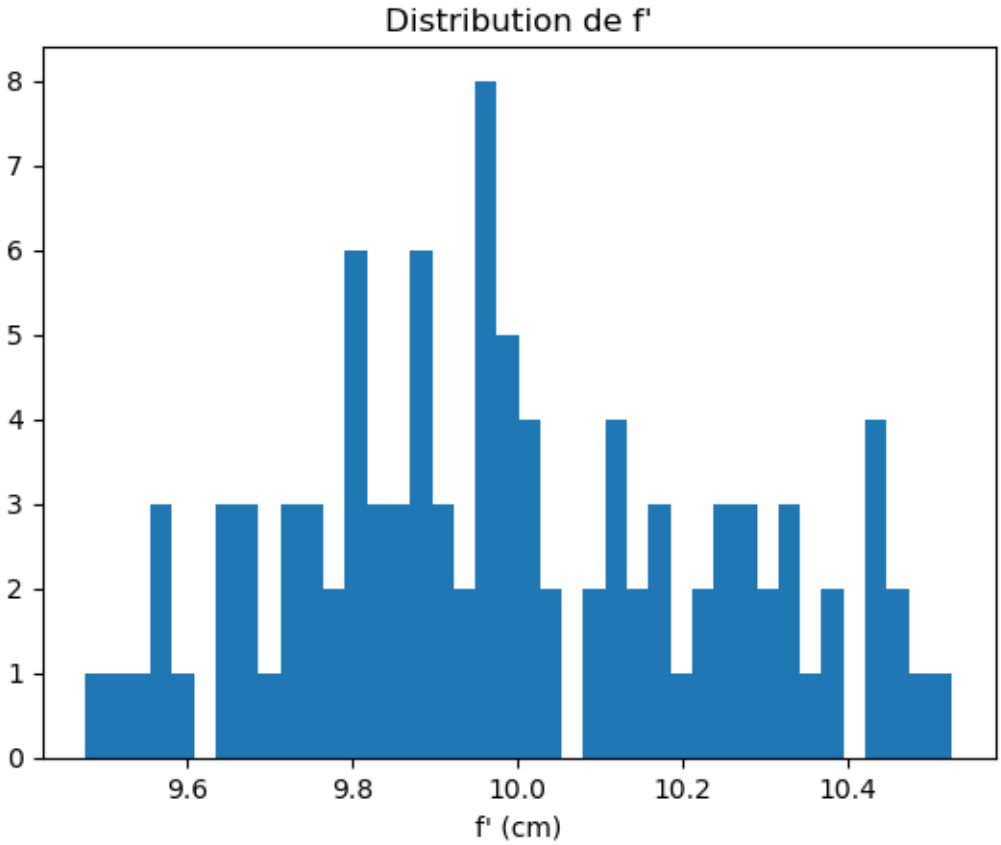
Utiliser la fonction `hist` de la bibliothèque `matplotlib.pyplot` pour afficher un histogramme des valeurs de f' comprenant 40 boîtes. La figure comportera un

titre et un libellé pour les abscisses. On pourra pour cela utiliser la fonction `matplotlib.pyplot.hist`, dont on donne la documentation ci-dessous.

`matplotlib.pyplot.hist` Paramètres : - `x` : liste d'entrée. - `bins` : si bins est un entier, il définit le nombre de boîtes de largeur égale.

[4]:
```
""" Importation des bibliothèques """
import matplotlib.pyplot as plt

""" Affichage de l'histogramme """
plt.close()
plt.hist(f, bins = 40)              # histogramme avec 40 boîtes
plt.title(r"Distribution de f'")    # titre
plt.xlabel(r"f' (cm)")              # libellé des abscisses
plt.show()                          # afficher la figure
```



Question 4

Déterminer et afficher la valeur moyenne et l'écart-type de la liste f. En déduire le résultat de mesure de f' avec son incertitude-type. On pourra utiliser les fonctions `np.mean` et `np.std`.

[5]:
```
fmoy    = np.mean(f)        # valeur moyenne de la distribution de f'
Deltaf  = np.std(f)         # écart-type de la distribution de f'
print("f' = (%.1f ± %.1f) cm"%(fmoy,Deltaf))
```

```
f' = (10.0 ± 0.3) cm
```

Question 5

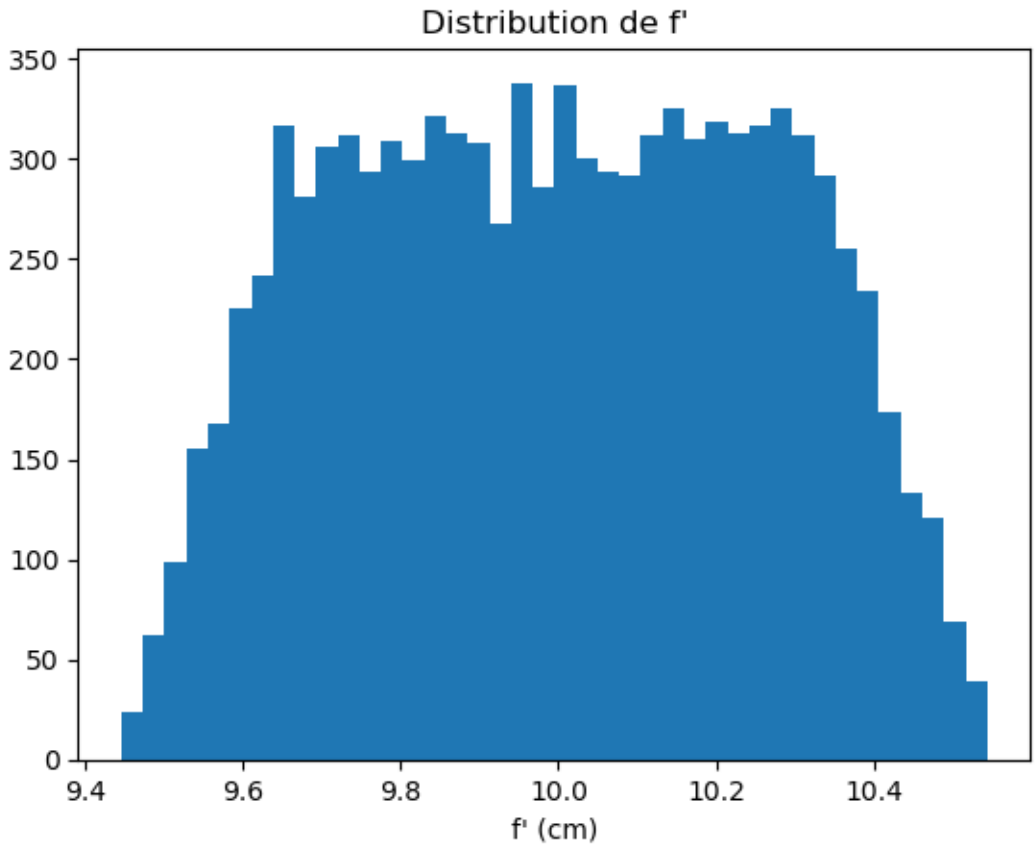
Refaire la simulation pour un nombre de tirage $N = 1000$, $N = 10000$ et $N = 100000$. Commenter l'allure de l'histogramme.

[6]:
```
""" Initialisations """
N = 10000 # nombre de tirages
f = []    # liste de valeurs de f'

""" Tirage de N valeurs de f' """
for i in range(N):
    X = np.random.uniform(x-Delta,x+Delta)  # tirage de X
    Y = np.random.uniform(y-Delta,y+Delta)  # tirage de Y
    f.append(focale(X,Y))                   # calcul de f'

""" Importation des bibliothèques """
import matplotlib.pyplot as plt

""" Affichage de l'histogramme """
plt.close()
plt.hist(f, bins = 40)              # histogramme avec 40 boîtes
plt.title(r"Distribution de f'")    # titre
plt.xlabel(r"f' (cm)")              # libellé des abscisses
plt.show()                          # afficher la figure
```


Distribution de f'
350
300
250
200
150
100
50
0
9.4
9.6
9.8
10.0
10.2
10.4
f' (cm)

[ ]:

CN5 - Propagation d'un paquet d'onde - Corrigé

April 6, 2025

1 CN5 - PROPAGATION D'UN PAQUET D'ONDE

1.1 A) Rappel théorique

On souhaite étudier la propagation d'une onde au cours du temps. Une onde plane complexe quelconque $\underline{s}(x,t)$ peut se mettre sous la forme

$$\boxed{\underline{s}(x,t) = \int_{-\infty}^{+\infty} \underline{A}(k) e^{i(\omega t - kx)} \mathrm{d}k,}$$

où $\underline{A}(k)$ désigne l'amplitude spectrale en vecteur d'onde k. Cette fonction est supposée être centrée en k_0 et de largeur typique Δk. Nous prendrons à titre d'exemple une fonction gaussienne réelle, de la forme

$$\underline{A}(k) = A(k) = A_0 e^{-(k-k_0)^2/\Delta k^2}.$$

1.2 B) Étude numérique

Question 1

Compléter le script suivant pour définir un tableau `numpy` représentant la fonction $A(k)$. On prendra $A_0 = 1$.

[1]:
```
import numpy as np
N    = 2**12      # Nombre de valeurs de k et de x
xmin = -20        # valeur minimale de x
xmax = 20         # valeur maximale de x
L    = xmax-xmin # Longueur de l'intervalle
X    = np.linspace(xmin, xmax, N)   # tableau des x
k    = np.linspace(0,2*np.pi*N/L,N) # tableau des k

k0     = 6
deltak = 2
A      = np.array([np.exp(-((k[i]-k0)/deltak)**2) for i in range(N)])
```

Question 2

Quelle est l'expression de $\underline{s}(x, t = 0)$ en fonction de $A(k)$? Compléter le script suivant pour définir une fonction `s_ini` qui renvoie tableau `numpy` représentant la fonction réelle $s(x, t = 0)$.

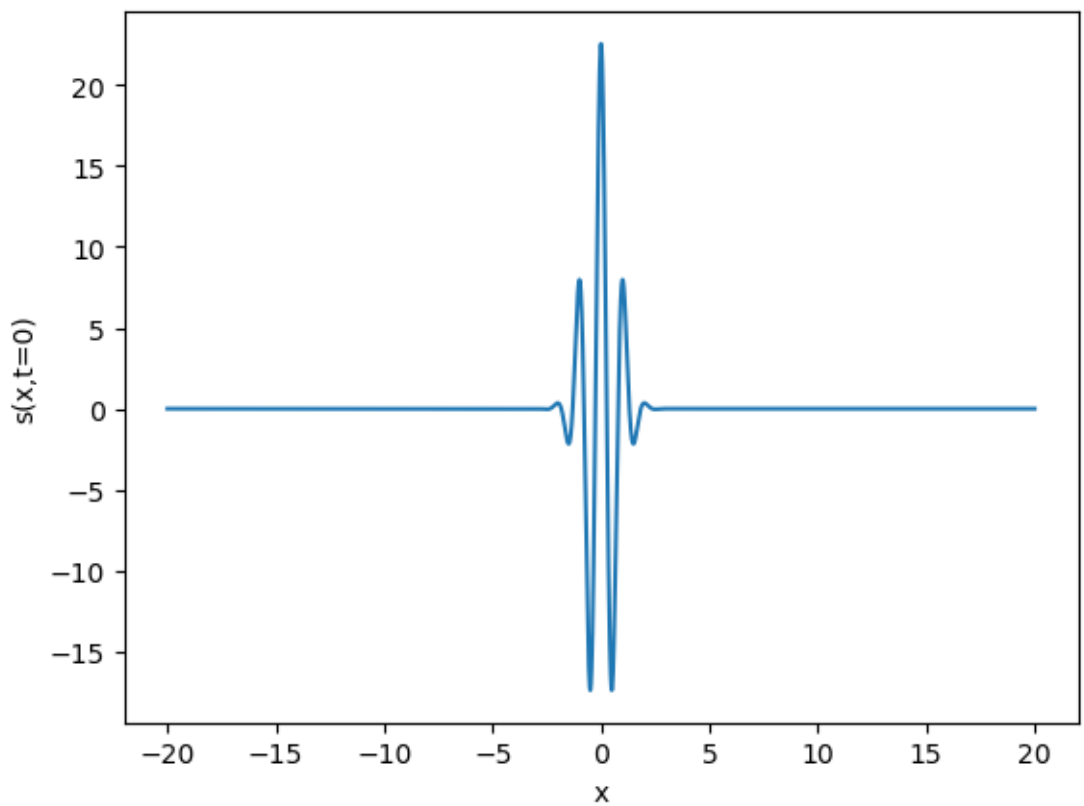
[2]:
```
def s_ini(k, X, A):
        s = np.zeros(len(X), dtype=np.complex_)
        for i in range(N):
            s += np.array(A[i]) * np.exp(1j * k[i] * X)
        return np.real(s)
```

Question 3

En s'aidant de la bibliothèque `matplotlib`, représenter la dépendance spatiale de l'onde initiale sur un graphe où x est porté en abscisse et $s(x, t = 0)$ en ordonnée. Commenter sur le lien entre la dépendance spatiale de l'onde initiale $s(x, t = 0)$ et l'amplitude spectrale $A(k)$.

[3]:
```
import matplotlib.pyplot as plt

s0 = s_ini(k, X, A)
plt.figure()
plt.plot(X, s0)
plt.xlabel('x')
plt.ylabel('s(x,t=0)')
plt.show()
plt.close()
```



Question 4

On suppose maintenant que $\Delta k \ll k_0$, de sorte qu'on puisse approximer la relation de dispersion comme $\omega(k) \simeq \omega_0 + (k - k_0)v_g$, où v_g représente la vitesse de groupe. Montrer alors que le paquet d'onde s'écrit

$$\boxed{\underline{s}(x,t) \simeq e^{i(\omega_0 t - k_0 x)} \int_{-\frac{\Delta k}{2}}^{+\frac{\Delta k}{2}} A(k+k_0) e^{-ik(x - v_g t)} \mathrm{d}k.}$$

Commenter la forme de cette fonction, et justifier que dans cette approximation un paquet d'onde ne se déforme pas au cours de sa propagation.

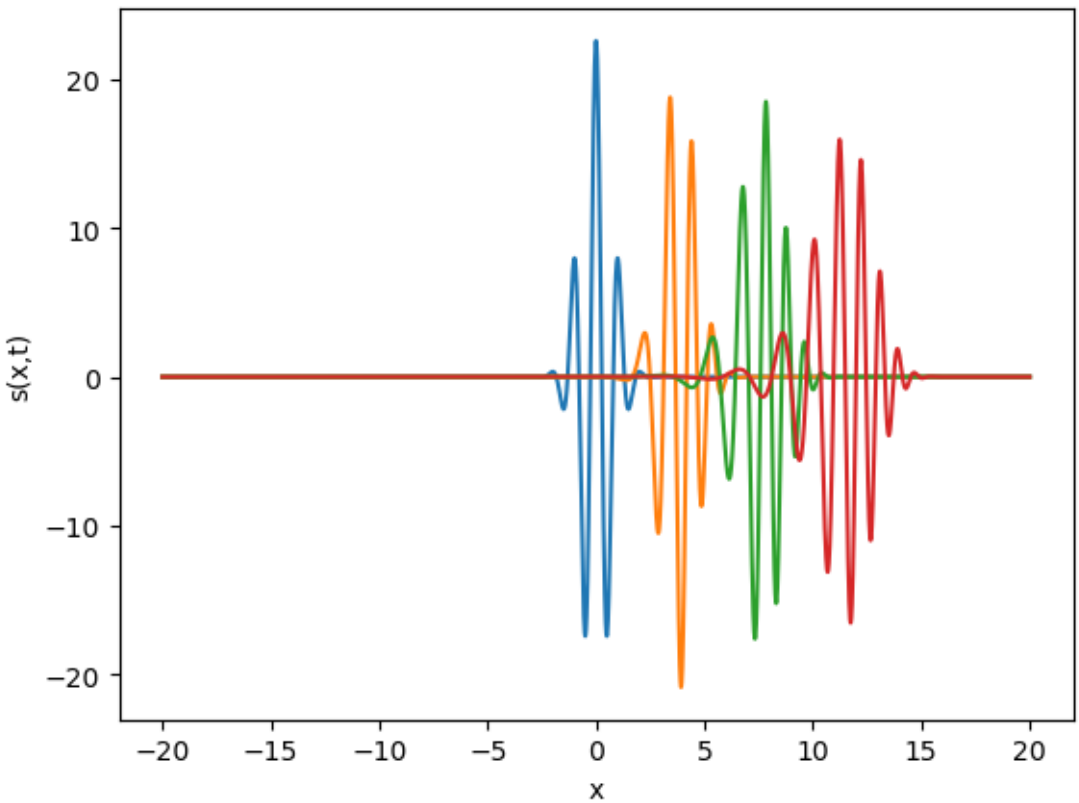
Question 5

On étudie la relation de dispersion $\omega(k)$ du plasma dans le régime haute fréquence $\omega > \omega_p$, où k est réel. Compléter le script suivant pour définir une fonction `s_temps` qui renvoie un tableau `numpy` représentant la fonction $s(x,t)$. Afficher le paquet d'onde pour la valeur $\Delta k = 2$ choisie précédemment, aux instant $t = 5$, $t = 10$, et $t = 15$. La linéarisation de la relation de dispersion est-elle valable avec les valeurs de Δk, c, ω_p et A_0 utilisées ici ? Observer la déformation du paquet d'onde.

[4]:
```
def omega(k):
    omegap = 5
    c = 1
    return (k**2*c**2+omegap**2)**0.5

def s_temps(k, X, A, t):
    s = np.zeros(len(X), dtype=np.complex_)
    for i in range(len(k)):
        s += np.array(A[i]) * np.exp(1j * k[i] * X) * np.exp(-1j * omega(k[i])␣
 ↪* t)
    return np.real(s)


plt.plot(X, s0)
for t in [5,10,15]:
    st = s_temps(k, X, A, t)
    plt.plot(X, st)
plt.xlabel('x')
plt.ylabel('s(x,t)')
plt.show()
plt.close()
```


20
10
0
−10
−20
s(x,t)
−20
−15
−10
−5
0
5
10
15
20
x

```
[ ]:
```